CONFIDENTIAL

SECURITY ASSESSMENT REPORT

HackerSavanna Bug Bounty Platform

www.hackersavanna.com

Report Date	2026-06-01
Researcher	Elvis Mbugua
Handle	`steiner254`
Platform	HackerSavanna
Program	HackerSavanna Vulnerability Disclosure Program (VDP)
Assessment Type	Blackbox
Report Version	1.0
Classification	**Confidential**

Findings at a Glance

ID	Title	Severity	CVSS v3.1
F-01	Firebase API Key Hardcoded in Client-Side JS Bundles	**Medium**	`5.3`
F-02	Unrestricted Firebase Account Registration via Exposed API Key	**High**	`7.5`
F-03	Password Reset Triggerable for Arbitrary Email Addresses	**Low**	`3.7`
F-04	SavannaHub Universal HTTP 200 / No Existence Validation	**Medium**	`5.3`
F-05	Sensitive Internal Endpoints in Public API Documentation	**Low**	`3.7`
F-06	DMARC Policy p=none — No Email Authentication Enforcement	**Low**	`3.1`

1. Executive Summary

This report documents the findings of an authorized penetration test conducted against the HackerSavanna bug bounty platform (www.hackersavanna.com) between 2026-05-30 and 2026-05-31. The assessment was conducted by Elvis Mbugua (handle: steiner254) under the HackerSavanna Vulnerability Disclosure Program (VDP).

Objective

Identify security weaknesses in the HackerSavanna web application, underlying Firebase backend, API infrastructure, and supporting components that could be exploited by an unauthenticated or authenticated attacker.

Testing Approach

Black-box to grey-box methodology, beginning with passive reconnaissance and escalating through active enumeration, client-side source code analysis of publicly served JavaScript bundles, Firebase API probing, and route/endpoint enumeration.

Overall Risk Posture

The platform demonstrates reasonable server-side access controls on its Firestore database and API endpoints. However, a significant exposure exists in the client-side JavaScript bundles, which embed the full Firebase project credentials. Combined with the absence of Firebase App Check or registration restrictions, any unauthenticated party can create accounts directly against Firebase Authentication, bypassing all application-level controls. No critical findings were identified; the highest severity is High (F-02).

2. Scope of Assessment

Asset	Type	IP / Host	Status
`www.hackersavanna.com`	Next.js / Vercel	`216.198.79.65, 64.29.17.65`	**Tested**
`hackersavanna.com`	Apex Domain	`216.198.79.1`	**Tested**
`hackersavana.firebaseapp.com`	Firebase Auth/Hosting	`GCP-managed`	**Tested**
`hackersavana.appspot.com`	Firebase Storage	`GCP-managed`	**Tested**
`Firestore (project: hackersavana)`	Firebase Firestore	`GCP-managed`	**Tested**
`identitytoolkit.googleapis.com`	Firebase Auth REST API	`GCP-managed`	**Tested**
`hackersavanna.firebaseio.com`	Firebase RTDB	`GCP-managed`	**Tested**
`/api/v1/*`	REST API Endpoints	`Vercel-served`	**Tested**
`/_next/static/chunks/*.js`	Client-Side JS Bundles	`Vercel CDN`	**Tested**

Assets Discovered During Assessment

Discovered Asset	Discovery Method
`Firebase Project ID: hackersavana`	JavaScript bundle extraction
`Firebase API Key: AIzaSyAPhk2Bhatz_j8dE6vvP7YXGuwYeEXLyAk`	JavaScript bundle extraction
`Firebase appId: 1:338136127186:web:177b09877642032f4dff0b`	JavaScript bundle extraction
`hackersavana.firebaseapp.com`	Firebase config extraction

`Internal API routes (backup, analytics, Paystack webhook)`	Public API documentation analysis
`MX Records: jellyfish.systems mail hosting`	DNS enumeration
`DNS: ns1–ns4.stableserver.net nameservers`	WHOIS / dnsrecon

3. Rules of Engagement

- Testing was conducted under the HackerSavanna Vulnerability Disclosure Program. Authorization is held by researcher handle steiner254.
- No destructive actions were intentionally performed. Test accounts used clearly identifiable addresses (e.g., pentestuser99@mailinator.com, test-[timestamp]@hackersavanna.com).
- Testing followed responsible disclosure principles. No data was exfiltrated, retained beyond verification, or shared with unauthorized parties.
- Evidence was collected solely for proving the existence and exploitability of reported vulnerabilities.
- All accounts created during testing should be considered disposable and can be deleted by the HackerSavanna security team upon review.
- No rate-limiting or denial-of-service testing was performed.

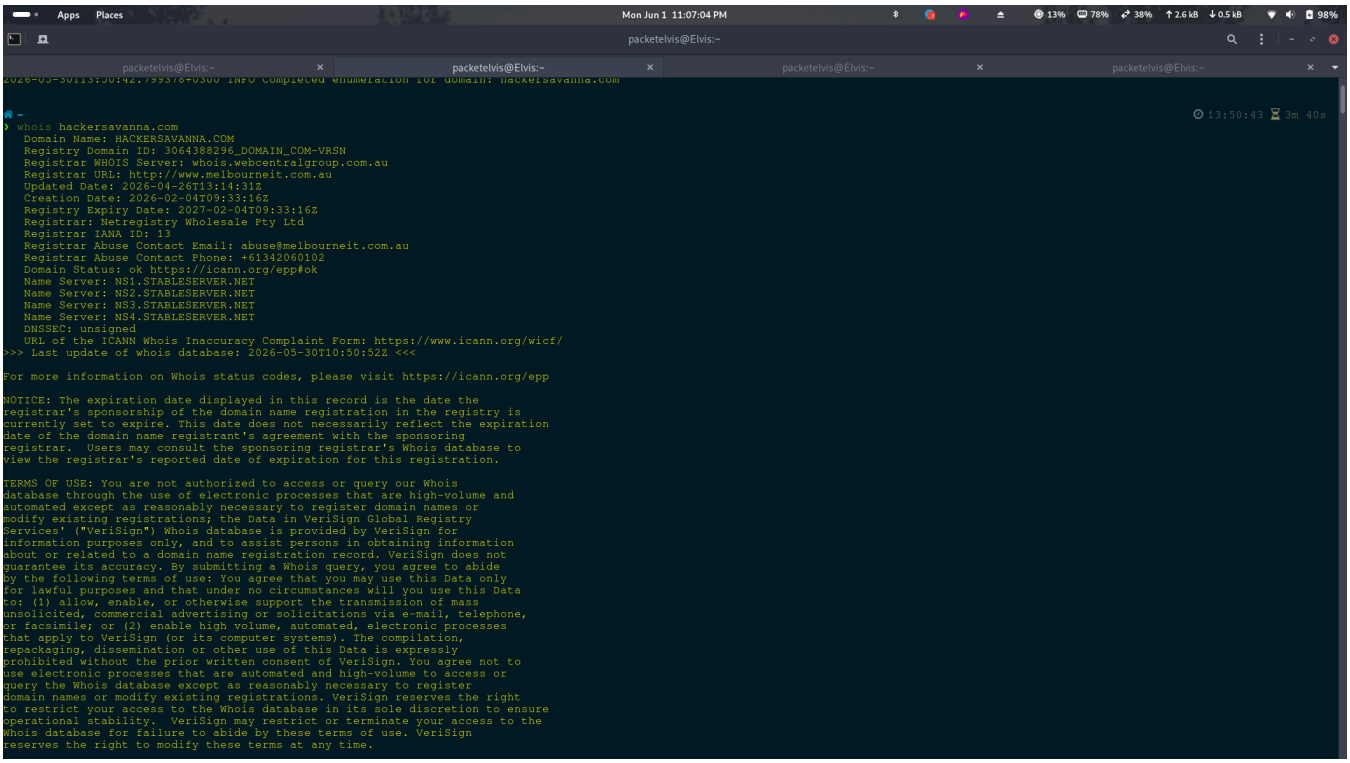
4. Methodology

Testing followed standard penetration testing phases aligned with the OWASP Testing Guide (OTGv4) and Penetration Testing Execution Standard (PTES).

Phase 1 — Passive Reconnaissance

Tools: *whois, crt.sh, web.archive.org, dnsrecon, httpx, whatweb*

- WHOIS confirmed domain registration date (2026-02-04), registrant location (Kenya), nameservers (stableserver.net).
- Certificate Transparency search via crt.sh confirmed only two live subdomains: hackersavanna.com and www.hackersavanna.com.
- Wayback Machine returned no archived content — new domain with no historical exposure.
- whatweb fingerprinted the tech stack: Vercel CDN, HTML5, HSTS, and identified security@hackersavanna.com in page metadata.



Phase 2 Enumeration

Tools: *dnsrecon, ffuf, curl, custom shell scripts*

- dnsrecon revealed SOA, NS, MX (jellyfish.systems), A, and TXT (SPF/DMARC) records.
- Subdomain brute-force confirmed www as the only web-facing subdomain; CNAME points to Vercel (d3fa430eb0a65c75.vercel-dns-017.com).
- ffuf directory fuzzing: .well-known/* paths return HTTP 403 no sensitive directories exposed.
- SavannaHub enumeration (/savannahub/1 through /savannahub/500): universal HTTP 200 for all IDs.
- Researcher profile enumeration (/r/{username}): all tested usernames returned HTTP 200.

Phase 3 Vulnerability Identification **Tools:** *curl, grep, custom shell scripts*

- Extracted 26 JavaScript chunk filenames from homepage HTML source.
- Downloaded all chunks and searched for Firebase config using minified-aware pattern: apiKey:"..." (unquoted key critical insight: minified JS omits quotes around object keys).
- Confirmed Firebase config in chunks 4198-6fcde7303608f4ff.js and app/layout-c3e8d87d9836f969.js.
- Identified NEXT_PUBLIC_FIREBASE env var reference confirming framework-level injection via Next.js public environment variables.
- API documentation page analyzed for internal endpoint disclosure.

Phase 4 Validation

Tools: *curl, Firebase Auth REST API, Firestore REST API*

- Firestore: All tested collections (users, reports, programs, companies, notifications) returned HTTP 403 PERMISSION_DENIED rules are appropriately restrictive.
- Firebase RTDB: {"error":"404 Not Found"} database not provisioned or not accessible.
- Firebase Auth: accounts:signUp accepted registration with no controls; two test accounts created with valid idToken values.

- Password reset OOB codes triggered for multiple addresses including security@hackersavanna.com and admin@hackersavanna.com.

5. Severity Ratings

Severity	Description	CVSS Range
Critical	Immediate system compromise; direct data breach path	9.0 – 10.0
High	Significant impact; authentication bypass, data exposure	7.0 – 8.9
Medium	Moderate risk; requires chaining or limited conditions	4.0 – 6.9
Low	Limited direct impact; configuration weaknesses	0.1 – 3.9
Informational	Security observation with no direct exploitability	N/A

6. Findings and Recommendations

F-01 Firebase API Key Hardcoded in Client-Side JavaScript Bundles

Severity: Medium | CVSS: **5.3 (AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)** | Asset: /_next/static/chunks/ — public Vercel CDN

Description

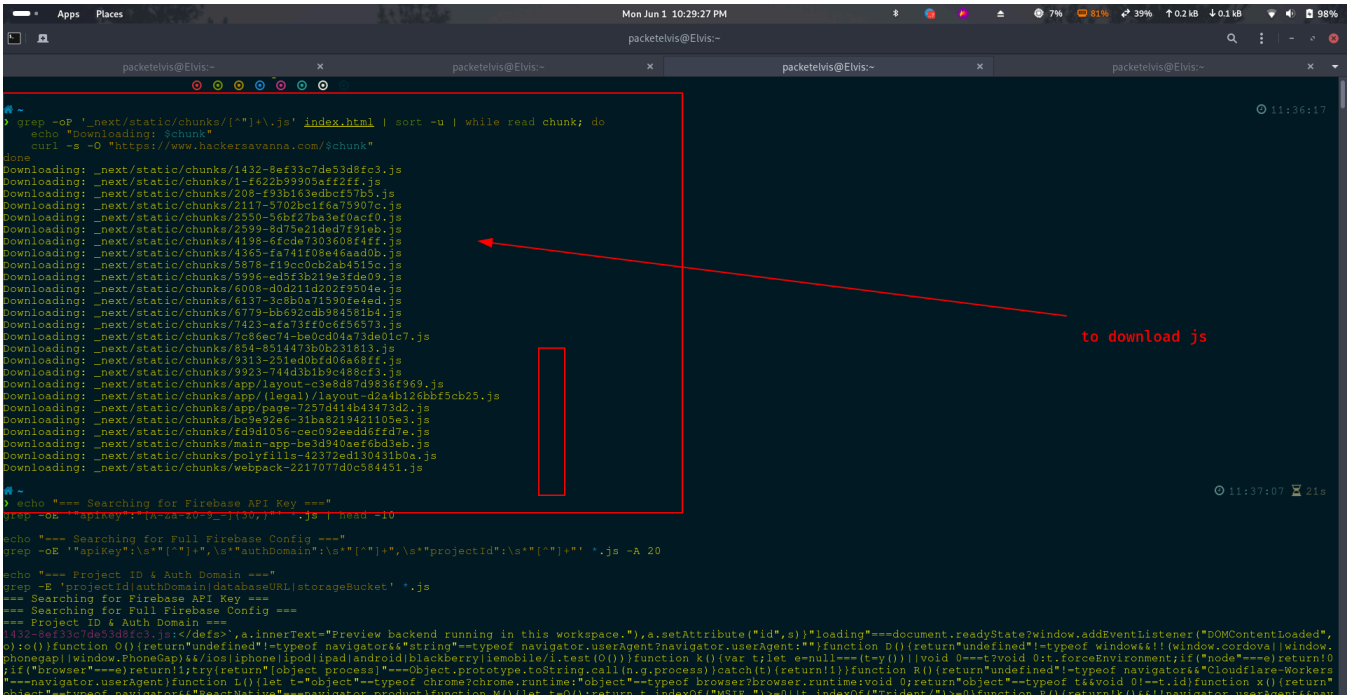
The HackerSavanna application is built on Next.js and deployed to Vercel. During JavaScript bundle analysis, the complete Firebase project configuration — including the API key, project identifiers, and application ID — was found hardcoded inside two publicly accessible JavaScript chunk files served from the Vercel CDN without authentication.

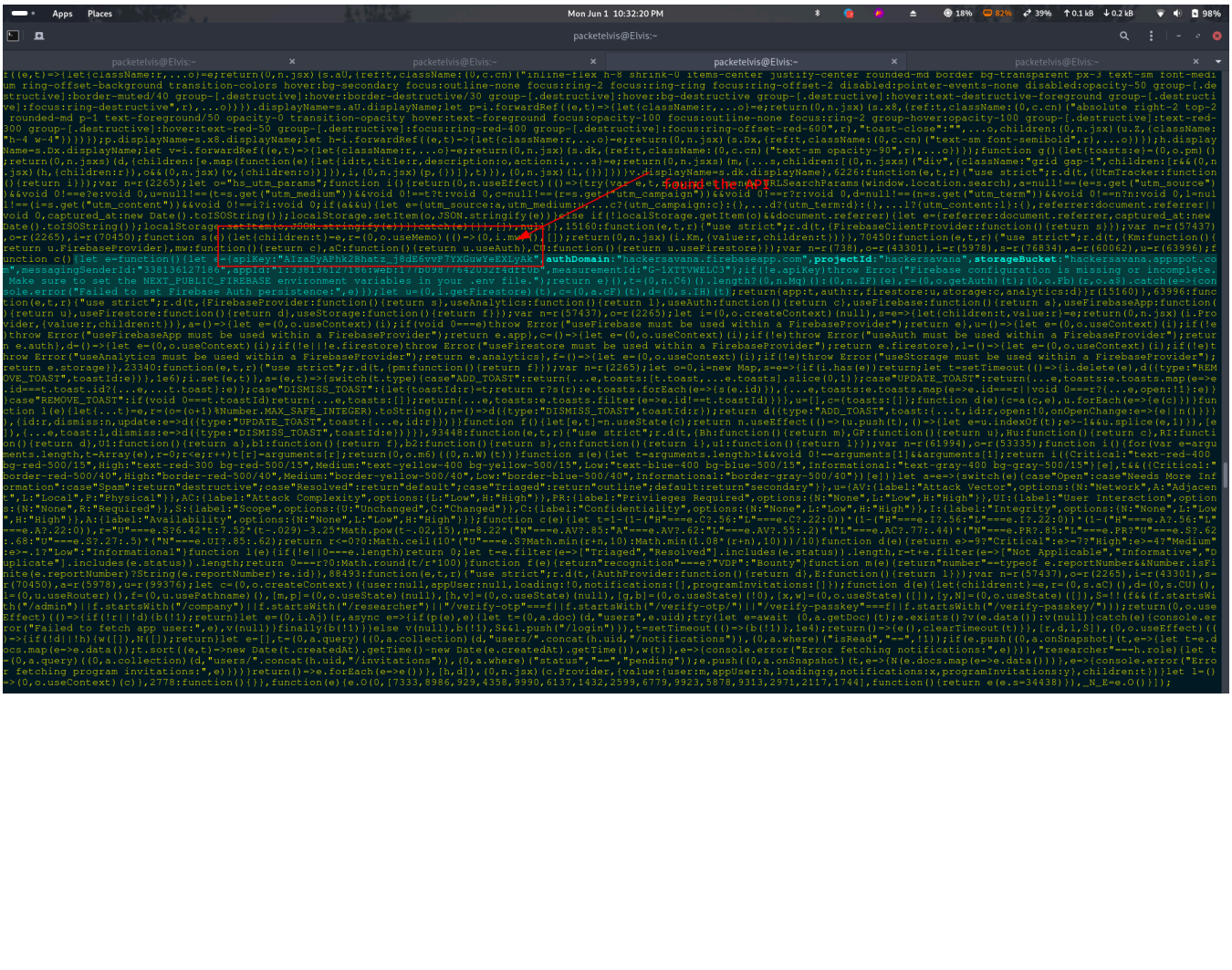
The NEXT_PUBLIC_FIREBASE environment variable pattern was also identified in app/layout-c3e8d87d9836f969.js, confirming credentials are injected via Next.js public environment variables — a mechanism explicitly designed to expose values to the browser. While Firebase API keys are not secret by design, their public exposure creates the attack surface exploited in F-02 and F-03.

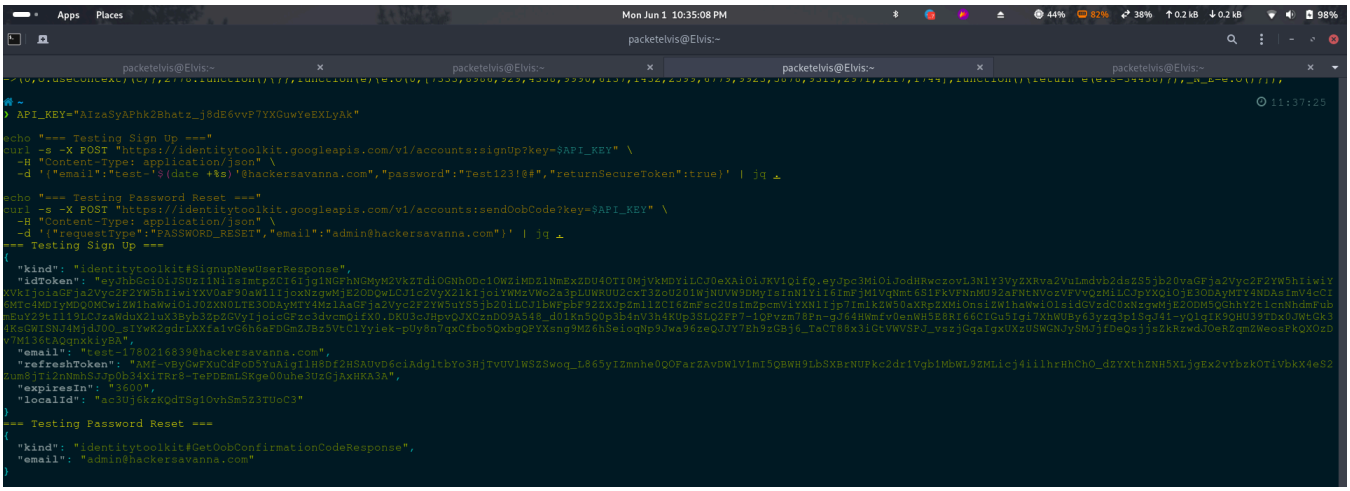
Extracted Firebase Configuration

Field	Value
apiKey	AIzaSyAPhk2Bhatz_j8dE6vvP7YXGuwYeEXLyAk
authDomain	hackersavana.firebaseapp.com
projectId	hackersavana
storageBucket	hackersavana.appspot.com
messagingSenderId	338136127186
appId	1:338136127186:web:177b09877642032f4dff0b
measurementId	G-1XTTVWELC3

Evidence







Reproduction Steps

```
# 1. Save homepage HTML
curl -s https://www.hackersavanna.com -o index.html

# 2. Extract chunk URLs
grep -oP '_next/static/chunks/[^"]+\.js' index.html | sort -u

# 3. Download target chunk
curl -s -O 'https://www.hackersavanna.com/_next/static/chunks/4198-6fcde7303608f4ff.js'

# 4. Extract Firebase config (minified/unquoted key pattern)
grep -oE 'apiKey:"[^"]{10,100}"' 4198-6fcde7303608f4ff.js
grep -oE 'projectId:"[^"]{3,50}"' 4198-6fcde7303608f4ff.js

# Full config recovered with no authentication.
```

Impact

Any unauthenticated external party can extract the full Firebase project configuration, enabling interaction with all Firebase services that accept client-initiated requests. If Firestore security rules are ever misconfigured even temporarily during a deployment the exposed API key provides immediate access to unprotected collections. The exposure is amplified significantly by F-02 (account registration) and F-03 (password reset abuse).

Recommendation

- Implement Firebase App Check to restrict API key use to verified application clients only. See: https://firebase.google.com/docs/app-check
- Restrict API key in Google Cloud Console: add HTTP Referrer restriction limiting use to *.hackersavanna.com.
- Review and enforce Firestore Security Rules using explicit identity and role checks.
- Enable Firebase Authentication Domain Allowlist to restrict OAuth redirect domains.

References: *OWASP A05:2021 – Security Misconfiguration* | *OWASP WSTG-CONF-05*

F-02 Unrestricted Firebase Account Registration via Exposed API Key

Severity: High | CVSS: **7.5 (AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:H/A:N)** | Asset: Firebase project: hackersavana — accounts:signUp endpoint

Description

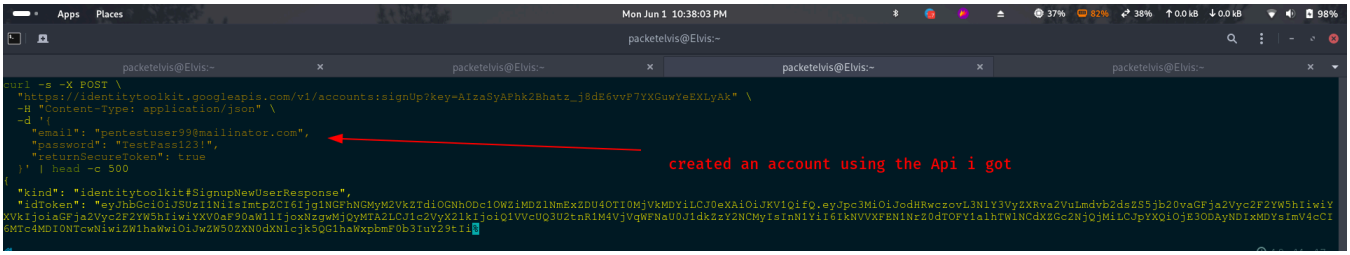
Using the Firebase API key recovered in F-01, any unauthenticated external party can create new Firebase Authentication accounts by sending a direct POST request to the Firebase Identity Toolkit REST API. This bypasses all application-level registration controls, including email verification, invite restrictions, CAPTCHA, or rate limiting enforced by the Next.js frontend.

Two test accounts were successfully created during testing:

- pentestuser99@mailinator.com (created 2026-05-30 18:41 EAT)
- test-1780216839@hackersavanna.com (created 2026-05-31 11:40 EAT)

Both registrations completed without any CAPTCHA challenge, email verification, domain restriction, or invite code requirement. Valid idToken, refreshToken, and localId values were returned for each, confirming the accounts are real and functional within Firebase Authentication.

Evidence



Reproduction Steps

```
# Obtain the API key (see F-01 reproduction steps)
API_KEY="AIzaSyAPhk2Bhatz_j8dE6vvP7YXGuwYeEXLyAk"

# Create a new Firebase account with no prior authentication
curl -s -X POST "https://identitytoolkit.googleapis.com/v1/accounts:signUp?key=$API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"email":"attacker@example.com","password":"AttackerPass123!","returnSecureToken":true}'

# Observe HTTP 200 with a valid idToken, refreshToken, and localId.
# No CAPTCHA. No invite code. No email verification. No domain restriction.
```

Impact

- Fraudulent Researcher Accounts: Unlimited accounts can be created programmatically, bypassing any vetting or invite-based registration, enabling access to researcher-only features and bounty claims.
- Frontend Control Bypass: Any registration-time controls in the Next.js UI (reCAPTCHA, invite codes, domain restrictions) are completely bypassed since the attack targets the Firebase REST API directly.
- Platform Trust Degradation: Automated account creation enables spam, fake leaderboard entries, and manipulation of program statistics.
- Lateral Movement Prerequisite: The obtained idToken may grant access to Firestore paths scoped to auth != null if rules are ever relaxed.
- @hackersavanna.com Domain Impersonation: Accounts with the platform's own domain (e.g., support@hackersavanna.com) can be created by external attackers.

Recommendation

- Implement Firebase App Check (Priority 1): Enforces that API calls originate from legitimate application instances. See: https://firebase.google.com/docs/app-check
- Deploy a Firebase Auth beforeCreate Blocking Function: Enforce custom registration logic (invitation validation, domain allowlist, CAPTCHA) server-side before account creation completes.

- Enforce Email Domain Restrictions: Restrict allowed sign-up domains via Firebase Console or blocking functions.
- Rate-Limit the signUp Endpoint: Implement per-IP or per-domain rate limiting via Firebase App Check or Cloud Armor.
- Delete Test Accounts: Remove pentestuser99@mailinator.com and test-1780216839@hackersavanna.com from the Firebase project immediately.

References: *OWASP A07:2021 – Identification and Authentication Failures | OWASP WSTG-IDNT-01*

F-03 Password Reset Triggerable for Arbitrary Email Addresses

Severity: Low | CVSS: **3.7** **(AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:L/A:N)** | Asset: identitytoolkit.googleapis.com/v1/accounts:sendOobCode

Description

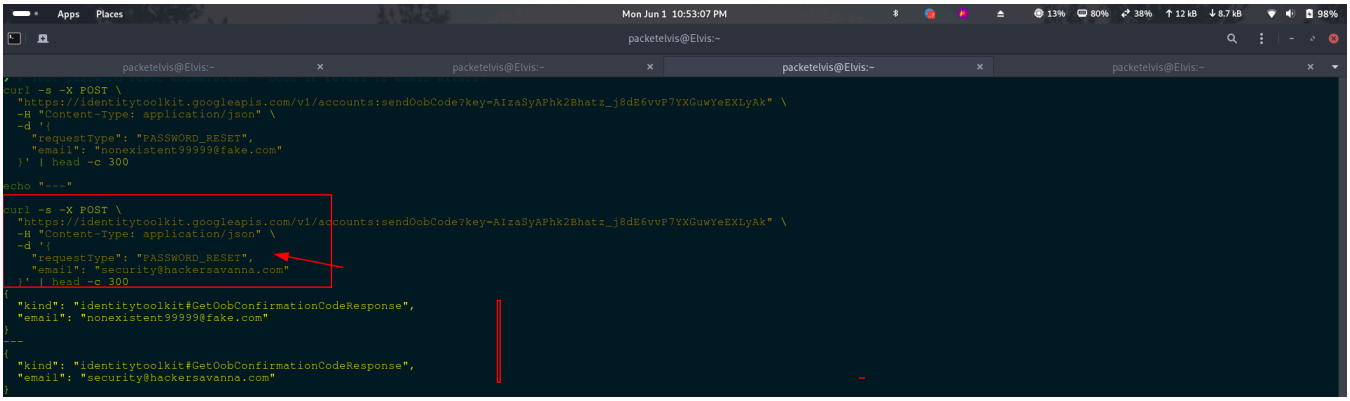
Using the exposed API key, an unauthenticated attacker can trigger Firebase password reset emails to any address by calling accounts:sendOobCode with requestType: PASSWORD_RESET. The endpoint returns an identical success response regardless of whether the target email address has a registered account.

Testing confirmed the same success response format for all of the following addresses:

- nonexistent99999@fake.com → GetOobConfirmationCodeResponse (no account exists)
- security@hackersavanna.com → GetOobConfirmationCodeResponse
- admin@hackersavanna.com → GetOobConfirmationCodeResponse

Note: The uniform response for both existing and non-existing emails is a privacy-positive design choice that prevents email enumeration — an attacker cannot determine whether an account exists from the response. The concern is the unrestricted ability to trigger unsolicited reset emails to arbitrary addresses with no rate limiting or application-level gating, which can be used to harass platform users.

Evidence



Reproduction Steps

```
API_KEY="AIzaSyAPhk2Bhatz_j8dE6vvP7YXGuwYeEXLyAk"

curl -s -X POST "https://identitytoolkit.googleapis.com/v1/accounts:sendOobCode?key=$API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"requestType":"PASSWORD_RESET","email":"target@example.com"}'

# Observe the success response returned for any email address submitted.
```

Impact

- An attacker can script automated password reset requests to platform user accounts sourced from the leaderboard or researcher profile pages, causing unsolicited reset emails and user confusion.
- Spam reset emails may lead users to distrust platform communications or click malicious links in spoofed follow-up emails.
- No rua= tag in the DMARC record (see F-06) means the platform also has no visibility into reset email spoofing attempts.

Recommendation

- Implement Firebase App Check to restrict Identity Toolkit calls to verified application clients.
- Rate-limit reset requests per IP via Cloud Armor or Vercel Edge Middleware.
- Add CAPTCHA to the application password reset flow before invoking the Firebase OOB endpoint.

References: *OWASP WSTG-AUTHN-09* | *OWASP A07:2021*

F-04 SavannaHub — Universal HTTP 200 Without Server-Side Existence Validation

Severity: Medium | CVSS: **5.3 (AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)** | Asset: https://www.hackersavanna.com/savannahub/{id} and /r/{username}

Description

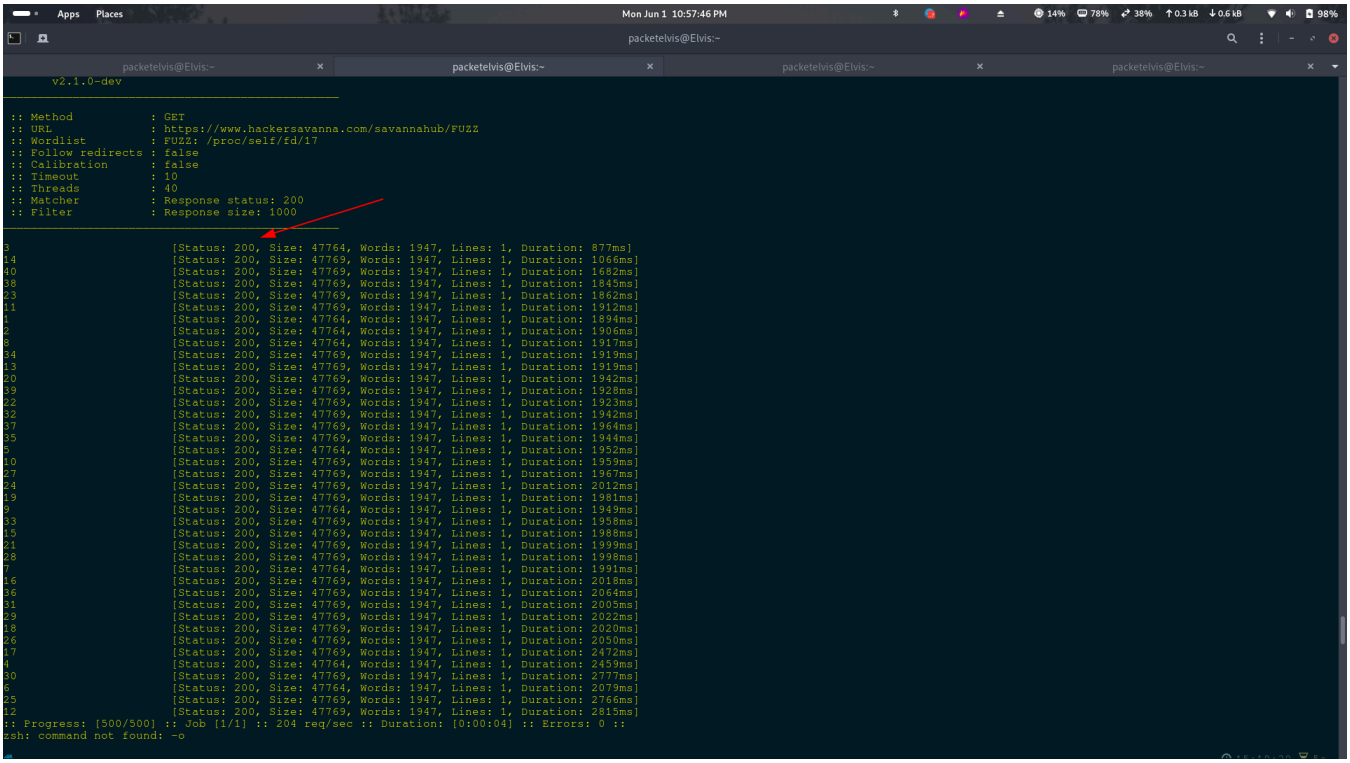
The SavannaHub section serves vulnerability report disclosure pages identified by sequential numeric IDs. Testing showed the application returns HTTP 200 for every numeric ID tested (1 through 500+), regardless of whether a corresponding report exists in the backend. All responses are Next.js server-rendered HTML shells of uniform size (~47,764–47,769 bytes).

The same behavior was confirmed on researcher profile pages — all tested usernames returned HTTP 200 regardless of registration status:

- Tested usernames: lia, justahacktitude, johns, M3rlin, innovara, Walaka12, jackie0x, kai7, toxic, darkbyte
- All 10 returned HTTP 200 with identical page shells

Note: Firestore security rules (confirmed appropriately protective in the validation phase) prevent actual report data from being loaded by unauthenticated visitors. However, the universal 200 response creates an enumeration surface and lacks appropriate server-side existence gating.

Evidence



Reproduction Steps

```
# SavannaHub ID enumeration
ffuf -u https://www.hackersavanna.com/savannahub/FUZZ -w <(seq 1 500) -mc 200

# Researcher profile enumeration
for user in test nonexistent random123 admin; do
  curl -s -o /dev/null -w "%{http_code}\n" https://www.hackersavanna.com/r/$user
done
# All return HTTP 200 regardless of username validity.
```

Impact

- Report ID enumeration allows mapping the range of active report IDs (IDs 1–40 appear to exist), enabling targeted attacks if Firestore rules are ever misconfigured.
- Sequential numeric IDs are trivially enumerable — an attacker does not need to guess.
- Researcher profile enumeration enables user account discovery for targeted phishing or credential-stuffing attacks.
- If access controls are relaxed in future, the enumerated IDs and usernames provide an immediate data extraction roadmap.

Recommendation

- Implement server-side report existence validation: Next.js server components should query the backend before rendering. Non-existent IDs should return HTTP 404; non-public reports should return HTTP 403.
- Apply the same pattern to /r/{username}: validate username existence server-side before rendering.
- Migrate to non-sequential, non-guessable report IDs (UUID v4 or random alphanumeric string).
- Enforce authentication gates: any undisclosed report should require a valid session before the page renders.

References: *OWASP A01:2021 – Broken Access Control | OWASP WSTG-ATHZ-04 (IDOR)*

F-05 Sensitive Internal API Endpoints in Public Documentation

Severity: Low | CVSS: **3.7** **(AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:N/A:N)** | Asset: https://www.hackersavanna.com/api-docs

Description

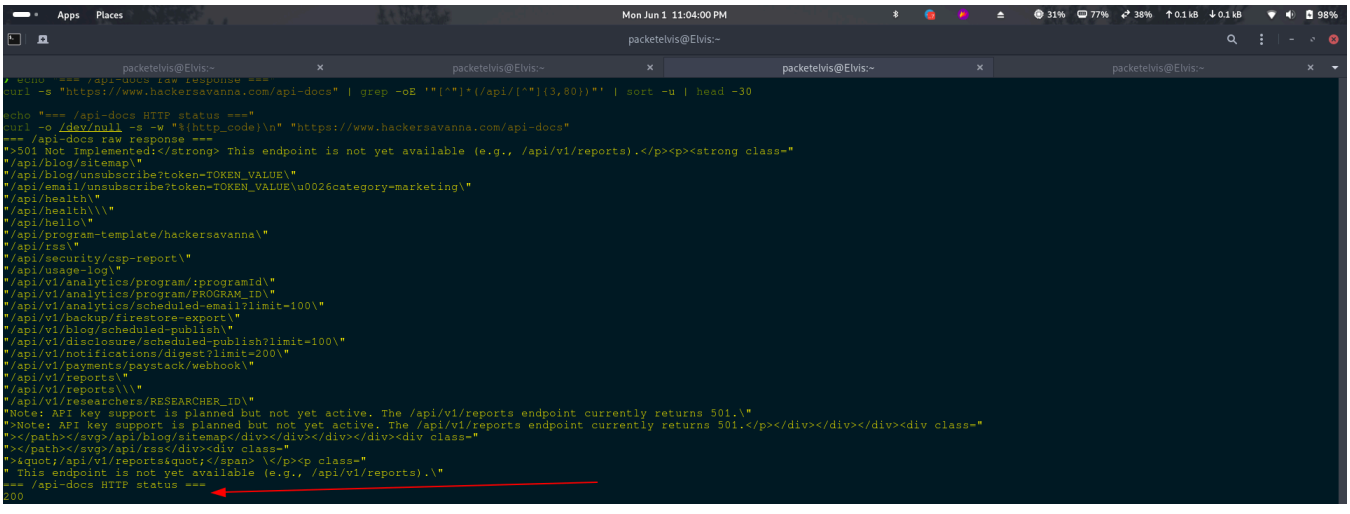
The platform's public API documentation page (HTTP 200, no authentication required) discloses the existence and path structure of internal administrative and operational endpoints not intended for public consumption.

Sensitive endpoints disclosed in public documentation:

Endpoint	Sensitivity	Risk
`/api/v1/backup/firestore-export`	**Critical**	Full DB backup trigger
`/api/v1/payments/paystack/webhook`	**High**	Payment processing injection
`/api/v1/analytics/scheduled-email?limit=100`	**Medium**	Internal analytics trigger
`/api/v1/notifications/digest?limit=200`	**Medium**	Internal notification digest
`/api/v1/disclosure/scheduled-publish?limit=100`	**Medium**	Report publication schedule
`/api/v1/blog/scheduled-publish`	**Low**	Blog publication automation
`/api/usage-log`	**Low**	Internal usage logging

The documentation also explicitly states "501 Not Implemented: This endpoint is not yet available (e.g., /api/v1/reports)" and "Note: API key support is planned but not yet active," disclosing the platform's development roadmap to external parties.

Evidence



Reproduction Steps

```
# Navigate to https://www.hackersavanna.com/api-docs (no auth required, returns HTTP 200)

# Extract all API paths:
curl -s "https://www.hackersavanna.com/api-docs" \
```

```
  | grep -oE '"[^"]*(/api/[^"]{3,80})"' | sort -u
```

Impact

- The /api/v1/backup/firestore-export endpoint, if lacking proper server-side auth, could expose the entire database on a single unauthenticated request.
- The Paystack webhook path enables targeted replay or injection attacks against the payment processing integration.
- Internal endpoint disclosure provides attackers a blueprint of backend architecture and technology stack.
- Development roadmap disclosure (501 endpoints, planned API key support) reveals the platform's security maturity and feature timeline.

Recommendation

- Remove internal endpoints from public documentation. Only expose endpoints intended for third-party developer integration.
- Enforce strict authentication on all admin endpoints: backup, analytics, notification, and scheduled-publish routes must require a server-side secret or service account credential.
- Protect the Paystack webhook: validate incoming requests using Paystack's HMAC signature (x-paystack-signature header) before processing.
- Implement IP allowlisting for operational endpoints: restrict to known internal IPs or Vercel Cron trusted origins only.

References: *OWASP A05:2021 – Security Misconfiguration | OWASP API Security API8:2019*

F-06 DMARC Policy p=none — No Email Authentication Enforcement

Severity: Low | CVSS: **3.1 (AV:N/AC:H/PR:N/UI:R/S:U/C:N/I:L/A:N)** | Asset: _dmarc.hackersavanna.com DNS record

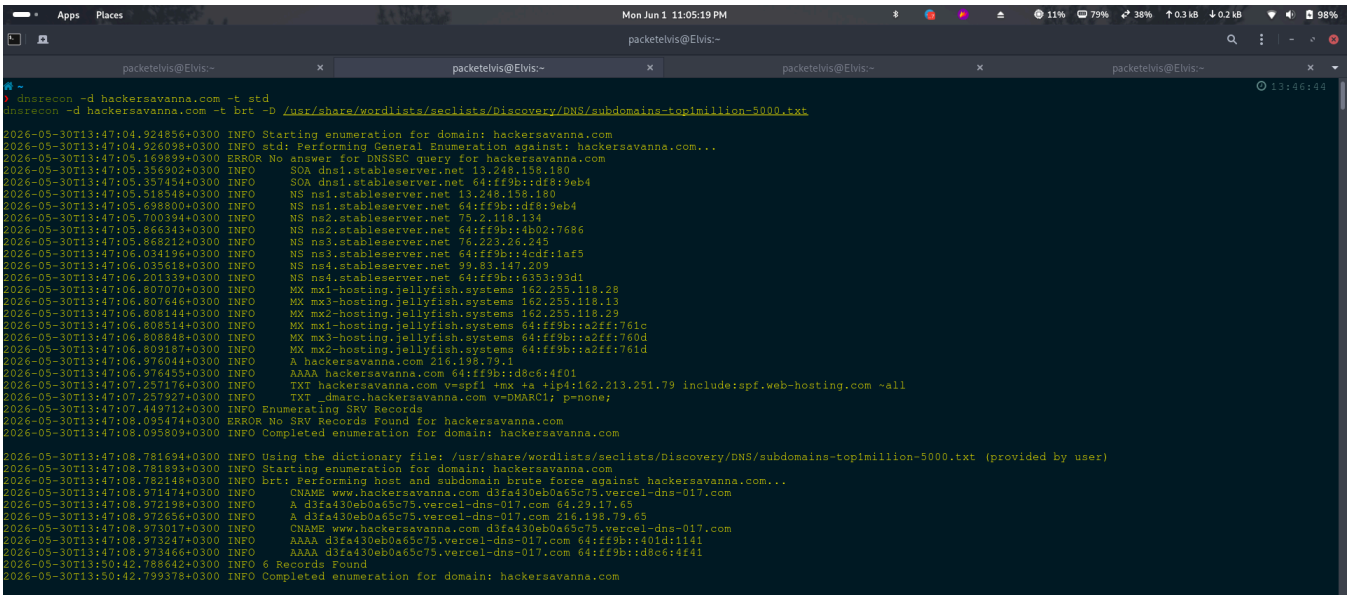
Description

DNS enumeration via dnsrecon revealed the hackersavanna.com domain has a DMARC record set to p=none. A DMARC policy of p=none means receiving mail servers take no action when emails fail DMARC authentication checks — making DMARC monitoring-only with no enforcement, leaving the domain vulnerable to email spoofing.

```
TXT _dmarc.hackersavanna.com   v=DMARC1; p=none;
TXT hackersavanna.com          v=spf1 +mx +a +ip4:162.213.251.79 include:spf.web-hosting.com ~all
```

The SPF record also uses a softfail (~all) rather than hardfail (-all), meaning unauthorized senders receive a softfail result rather than a hard rejection. This is particularly sensitive for a security platform whose communications include bounty payment notifications, vulnerability disclosure alerts, and account security emails — all high-value targets for spoofing. No rua= reporting tag was observed, suggesting the organization may have no visibility into ongoing spoofing attempts.

Evidence



Reproduction Steps

```
dnsrecon -d hackersavanna.com -t std
# Observe the TXT record section:
# TXT hackersavanna.com    v=spf1 ... ~all
# TXT _dmarc.hackersavanna.com   v=DMARC1; p=none;
```

Impact

- Emails spoofed as @hackersavanna.com (e.g., security@hackersavanna.com, noreply@hackersavanna.com) can reach researcher and company inboxes without being filtered or quarantined.
- Phishing campaigns targeting HackerSavanna users can be conducted with authentic-looking sender addresses.
- No rua= tag means the organization has no visibility into ongoing spoofing attempts.
- Combined with the platform's trust position (security researchers expect official emails about vulnerabilities and payments), spoofed emails are likely to be acted upon.

Recommendation

- Escalate DMARC to p=quarantine immediately, then p=reject: start with v=DMARC1; p=quarantine; pct=50; rua=mailto:dmarc-reports@hackersavanna.com — monitor for 2–4 weeks, then escalate to p=reject; pct=100.
- Harden SPF to -all (hardfail): change ~all to -all once all legitimate sending sources are confirmed.
- Implement DKIM signing: ensure all outbound emails are DKIM-signed with a key published in DNS.
- Add DMARC reporting tags: include rua= and ruf= for aggregate and forensic failure visibility.

References: *OWASP WSTG-CONF-10 | RFC 7489 – DMARC | NIST SP 800-177*

7. Conclusion

This security assessment of www.hackersavanna.com identified six findings spanning High to Low severity. No Critical findings were identified. The backend Firestore database and core API endpoints demonstrate appropriate access controls, representing a solid security foundation. The primary risk is concentrated at the Firebase

Authentication layer, where the combination of client-side credential exposure and unrestricted account registration creates a path for unauthorized account creation at scale.

Total Findings	6
Highest Severity	High (F-02)
Finding Breakdown	1 High | 2 Medium | 3 Low
Positive Controls Confirmed	Firestore rules (403 all collections), RTDB not exposed, HSTS, X-Frame-Options: DENY, CSP reporting endpoint active

Priority Remediation Actions

Priority	Timeline	Action
1	Immediate	Deploy Firebase App Check to restrict all Authentication API access to verified application clients (addresses F-01, F-02, F-03)
2	Immediate	Implement Firebase Auth beforeCreate Blocking Function to enforce server-side registration controls, domain allowlist, and CAPTCHA (addresses F-02)
3	Short-term	Add server-side existence validation to /savannahub/{id} and /r/{username}; migrate to UUID-based report IDs (addresses F-04)
4	Short-term	Escalate DMARC from p=none to p=quarantine and begin aggregate reporting; harden SPF to -all (addresses F-06)
5	Medium-term	Remove internal operational endpoints from public API documentation; enforce strict auth on all admin routes (addresses F-05)
6	Medium-term	Restrict Firebase API key to *.hackersavanna.com referrers in Google Cloud Console (reduces F-01 blast radius)